
TRÍ TUỆ NHÂN TẠO

Machine Learning

Phần 3 — Statistics

Biên soạn:

ĐẶNG TẤN QUÍ

SĐT: 0377 122 210

2026

Mục lục

Lời nói đầu	3
1 Statistics — Thống kê trong học máy	4
1.1 Statistics là gì?	4
1.2 Các khái niệm thống kê cơ bản cho ML	4
1.3 Hai loại thống kê	5
1.3.1 Thống kê mô tả (Descriptive Statistics)	5
1.3.2 Thống kê suy luận (Inferential Statistics)	6
2 Mean, Median, Mode — Trung bình, trung vị, mode	6
2.1 Mean (Trung bình)	6
2.2 Median (Trung vị)	7
2.3 Mode (Mode)	7
2.4 Triển khai Python	7
3 Standard Deviation — Độ lệch chuẩn	8
3.1 Ví dụ 1 — NumPy	8
3.2 Ví dụ 2 — Iris (Pandas)	9
4 Percentiles — Phân vị	9
4.1 Tính percentile với NumPy	10
4.2 Tính percentile với Pandas	10
5 Data Distribution — Phân phối dữ liệu	10
5.1 Phân phối chuẩn (Normal / Gaussian)	11
5.2 Phân phối lệch (Skewed)	12
5.3 Phân phối đều (Uniform)	12
5.4 Phân phối hai đỉnh (Bimodal)	13
6 Skewness and Kurtosis — Độ lệch và độ nhọn	14
6.1 Ví dụ Python (SciPy)	15
7 Bias and Variance — Độ lệch và phương sai mô hình	15
7.1 Bias là gì?	16
7.2 Variance là gì?	16
7.3 Đánh đổi Bias–Variance	17
7.4 Ví dụ Python	18
8 Hypothesis — Giả thuyết trong học máy	20
8.1 Giả thuyết trong Machine Learning	20
8.2 Hàm giả thuyết h	20
8.3 Không gian giả thuyết $\mathcal{H}$	21
8.4 Hai loại giả thuyết (kiểm định thống kê)	21
8.5 Kiểm định giả thuyết trong ML	21
8.6 Tìm giả thuyết tốt nhất	21
8.7 Tính chất giả thuyết tốt	22

Lời nói đầu

Nguồn

Tài liệu biên soạn và dịch đầy đủ từ Tutorialspoint Machine Learning — mục Statistics: https://www.tutorialspoint.com/machine_learning/machine_learning_statistics.htm.

Cách dùng

- Đọc sau phần Basic và Data Visualization.
- Ví dụ Python: NumPy, Pandas, SciPy, scikit-learn, statsmodels.

Đặng Tấn Quý

1 Statistics — Thống kê trong học máy

Vai trò của thống kê

Thống kê là công cụ then chốt trong học máy vì giúp hiểu pattern ẩn trong dữ liệu. Nó cung cấp phương pháp mô tả, tóm tắt và phân tích dữ liệu.

1.1 Statistics là gì?

Định nghĩa

Thống kê là nhánh toán học về thu thập, phân tích, diễn giải và trình bày dữ liệu. Nó cung cấp các phương pháp và kỹ thuật phân tích dữ liệu và rút kết luận.

Nền tảng cho ML

Thống kê là nền tảng học máy: phân tích và trực quan hoá để tìm pattern. Ứng dụng: xác thực mô hình, làm sạch dữ liệu, chọn mô hình, đánh giá hiệu năng, ...

1.2 Các khái niệm thống kê cơ bản cho ML

Danh sách khái niệm quan trọng

- **Trung bình (Mean), Trung vị (Median), Mode:** đo xu hướng trung tâm.
- **Phương sai (Variance), Độ lệch chuẩn (Standard deviation):** độ phân tán quanh mean.
- **Phân vị (Percentiles):** giá trị dưới đó một tỷ lệ % quan sát rơi vào.
- **Phân phối dữ liệu (Data distribution):** cách điểm dữ liệu trải trên miền giá trị.
- **Độ lệch (Skewness), Độ nhọn (Kurtosis):** độ lệch và độ nhọn của phân phối.
- **Sai số (Bias), Phương sai (Variance):** nguồn sai số dự đoán mô hình.
- **Giả thuyết (Hypothesis):** giả thuyết / lời giải đề xuất cho bài toán.
- **Hồi quy tuyến tính (Linear Regression):** dự đoán giá trị của một biến dựa trên giá trị của một biến khác.
- **Hồi quy logistic (Logistic Regression):** ước tính xác suất xảy ra của một sự kiện.
- **Phân tích thành phần chính (PCA):** giảm số chiều của dữ liệu bằng cách tìm ra các thành phần chính (principal components) của dữ liệu.

1.3 Hai loại thống kê

Thống kê mô tả

Tập quy tắc/phương pháp dùng để **mô tả hoặc tóm tắt** đặc trưng của dataset.

Thống kê suy luận

Đối phó với **dự đoán và suy luận** về quần thể dựa trên mẫu dữ liệu.

1.3.1 Thống kê mô tả (Descriptive Statistics)

Nội dung

Nhánh thống kê tóm tắt và phân tích dữ liệu: mean, median, mode, variance, standard deviation. Giúp hiểu xu hướng trung tâm, độ biến thiên và hình dạng phân phối.

Ứng dụng trong ML

Tóm tắt dữ liệu, nhận diện outlier, phát hiện pattern. Ví dụ: dùng mean và standard deviation để mô tả phân phối một feature.

Tính thống kê mô tả với NumPy và Pandas

```
import numpy as np
import pandas as pd

data = np.array([1, 2, 3, 4, 5])
df = pd.DataFrame(data, columns=["Values"])
print(df.describe())
```

Output (tóm tắt gồm count, mean, std, min, các phân vị, max):

	Values
count	5.000000
mean	3.000000
std	1.581139
min	1.000000
25%	2.000000
50%	3.000000
75%	4.000000
max	5.000000

1.3.2 Thống kê suy luận (Inferential Statistics)

Nội dung

Nhánh thống kê dự đoán và suy luận về quần thể từ mẫu: kiểm định giả thuyết, khoảng tin cậy, hồi quy.

Ứng dụng trong ML

Dự đoán trên dữ liệu mới dựa trên dữ liệu hiện có — ví dụ hồi quy giá nhà theo số phòng, diện tích, ...

Hồi quy OLS với statsmodels

```
import statsmodels.api as sm
import numpy as np

X = np.array([1, 2, 3, 4, 5])
y = np.array([2, 4, 6, 8, 10])

X = sm.add_constant(X)
model = sm.OLS(y, X).fit()

print(model.summary())
```

Output (trích): bảng hệ số, std err, t , $P > |t|$, R^2 ; với dữ liệu tuyến tính hoàn hảo, hệ số $x_1 \approx 2$.

2 Mean, Median, Mode — Trung bình, trung vị, mode

Ba đo xu hướng trung tâm

Mean (trung bình), Median (trung vị) và Mode (mode) là các đại lượng mô tả **xu hướng trung tâm** của dataset. Trong ML, chúng giúp hiểu phân phối dữ liệu và nhận diện outlier.

2.1 Mean (Trung bình)

Mean

Mean là **giá trị trung bình**: cộng tất cả giá trị rồi chia cho số quan sát. Mean nhạy với **outlier** — giá trị cực đoan kéo mean mạnh.

Python

Dùng `np.mean()` trong NumPy.

2.2 Median (Trung vị)

Median

Median là giá trị **ở giữa** sau khi sắp xếp. Nếu số phần tử chẵn, median là trung bình của hai giá trị giữa. Median **ít bị ảnh hưởng** bởi outlier hơn mean.

Python

Dùng `np.median()`.

2.3 Mode (Mode)

Mode

Mode là giá trị **xuất hiện nhiều nhất**. Nếu nhiều giá trị cùng tần suất cao nhất, dataset có thể lưỡng cực (bimodal), tam cực (trimodal) hoặc đa cực (multimodal). Mode hữu ích khi tìm giá trị phổ biến nhất; kém phù hợp khi dữ liệu trải rộng hoặc không có giá trị lặp.

Python

SciPy có `mode()`; Pandas: `Series.mode()`.

2.4 Triển khai Python

Bảng lương mẫu

```
import numpy as np
import pandas as pd
# create a sample salary table
salary = pd.DataFrame({
    'employee_id': ['001', '002', '003', '004', '005', '006', '007',
                    '008', '009', '010'],
    'salary': [50000, 65000, 55000, 45000, 70000, 60000, 55000, 45000,
              80000, 70000]
})

# calculate mean
mean_salary = np.mean(salary['salary'])
print('Mean salary:', mean_salary)

# calculate median
median_salary = np.median(salary['salary'])
print('Median salary:', median_salary)

# calculate mode
mode_salary = salary['salary'].mode()[0]
print('Mode salary:', mode_salary)
```

Output:

```
Mean salary: 59500.0
Median salary: 57500.0
Mode salary: 45000
```

Đọc kết quả

Mean cao hơn median một chút (ảnh hưởng lương 70k–80k); mode 45000 vì hai nhân viên cùng mức lương đó.

3 Standard Deviation — Độ lệch chuẩn

Standard deviation

Độ lệch chuẩn đo mức **phân tán** (variation/dispersion) của các giá trị quanh mean. Trong ML, nó mô tả độ trải của dataset và hỗ trợ phát hiện outlier.

Công thức

Độ lệch chuẩn là căn bậc hai của phương sai — trung bình bình phương độ lệch so với mean:

$$\sigma = \sqrt{\frac{\sum (x - \mu)^2}{N}}$$

trong đó σ là độ lệch chuẩn, x là điểm dữ liệu, μ là mean, N là số điểm.

Ứng dụng

Tài chính: đo biến động giá cổ phiếu; xử lý ảnh: phát hiện nhiễu, ...

3.1 Ví dụ 1 — NumPy

Độ lệch chuẩn mảng đơn giản

```
import numpy as np

data = np.array([1, 2, 3, 4, 5, 6])
std_dev = np.std(data)

print('Standard deviation:', std_dev)
```

Output:

```
Standard deviation: 1.707825127659933
```


3.2 Ví dụ 2 — Iris (Pandas)

Độ lệch chuẩn từng cột Iris

```
import pandas as pd
from sklearn.datasets import load_iris

iris = load_iris()
iris_df = pd.DataFrame(
    iris.data,
    columns=['sepal length', 'sepal width', 'petal length', 'petal width']
)

std_devs = iris_df.std()
print('Standard deviations:')
print(std_devs)
```

Output:

```
Standard deviations:
sepal length    0.828066
sepal width     0.433594
petal length    1.764420
petal width     0.763161
dtype: float64
```

Nhận xét

Độ lệch chuẩn `petal length` cao hơn hẳn các cột khác — feature này biến thiên mạnh, có thể hữu ích cho phân loại.

4 Percentiles — Phân vị

Percentile (phân vị)

Percentile là đại lượng cho biết giá trị **dưới đó** một tỷ lệ % quan sát trong nhóm rơi vào.

Ví dụ

- **Phân vị thứ 25 (Q1)**: 25% quan sát nằm dưới giá trị này.
- **Phân vị thứ 75 (Q3)**: 75% quan sát nằm dưới giá trị này.

Percentile tóm tắt phân phối và giúp nhận diện outlier; thường dùng trong tiền xử lý và EDA.

Thư viện Python

NumPy (`np.percentile`) và Pandas (`quantile`).

4.1 Tính percentile với NumPy

NumPy

```
import numpy as np

data = np.array([1, 2, 3, 4, 5])
p25 = np.percentile(data, 25)
p75 = np.percentile(data, 75)
print('25th percentile:', p25)
print('75th percentile:', p75)
```

Output:

```
25th percentile: 2.0
75th percentile: 4.0
```

4.2 Tính percentile với Pandas

Pandas Series

```
import pandas as pd

data = pd.Series([1, 2, 3, 4, 5])
p25 = data.quantile(0.25)
p75 = data.quantile(0.75)

print('25th percentile:', p25)
print('75th percentile:', p75)
```

Output:

```
25th percentile: 2.0
75th percentile: 4.0
```

5 Data Distribution — Phân phối dữ liệu

Phân phối dữ liệu

Trong ML, phân phối dữ liệu là cách các điểm dữ liệu **trải rộng** trên dataset. Hiểu phân phối ảnh hưởng lớn đến hiệu năng thuật toán.

Đại lượng mô tả

Mean, median, mode, độ lệch chuẩn, phương sai mô tả xu hướng trung tâm, độ trải và hình dạng.

5.1 Phân phối chuẩn (Normal / Gaussian)

Normal distribution

Phân phối xác suất liên tục dạng **chuông**, đối xứng quanh mean; hai tham số: mean μ và độ lệch chuẩn σ .

Ứng dụng trong ML

Mô hình hoá sai số trong hồi quy tuyến tính; cơ sở kiểm định giả thuyết và khoảng tin cậy.

Quy tắc 68–95–99.7

Khoảng $[\mu - \sigma, \mu + \sigma]$ chứa $\approx 68\%$ quan sát; $\pm 2\sigma: \approx 95\%$; $\pm 3\sigma: \approx 99,7\%$.

Sinh mẫu và vẽ PDF (scipy.stats)

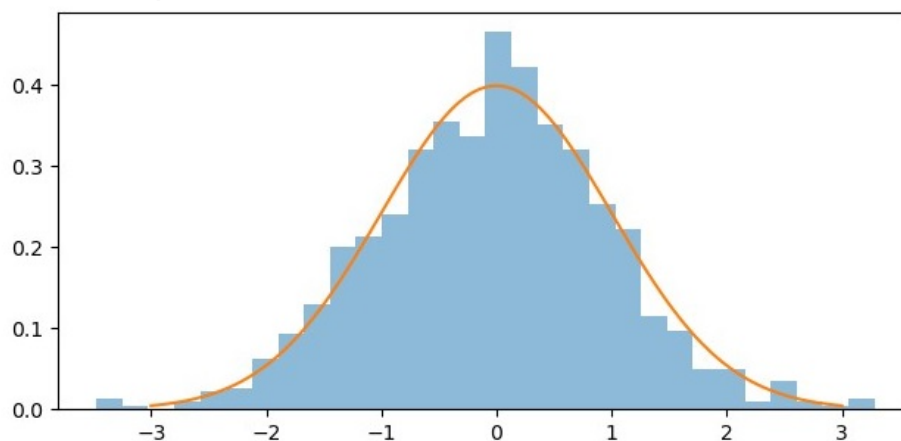
```
import numpy as np
from scipy.stats import norm
import matplotlib.pyplot as plt

mu = 0
sigma = 1
sample = np.random.normal(mu, sigma, 1000)

x = np.linspace(mu - 3*sigma, mu + 3*sigma, 100)
pdf = norm.pdf(x, mu, sigma)

plt.figure(figsize=(7.5, 3.5))
plt.hist(sample, bins=30, density=True, alpha=0.5)
plt.plot(x, pdf)
plt.show()
```

Output:



5.2 Phân phối lệch (Skewed)

Skewed distribution

Dataset không đối xứng quanh mean; phần lớn điểm dồn một phía, đuôi kéo dài phía kia.

- **Lệch trái** (negative skew): đuôi dài bên trái.
- **Lệch phải** (positive skew): đuôi dài bên phải.

Dữ liệu lệch có thể làm méo mô hình — cần chuẩn hoá hoặc biến đổi.

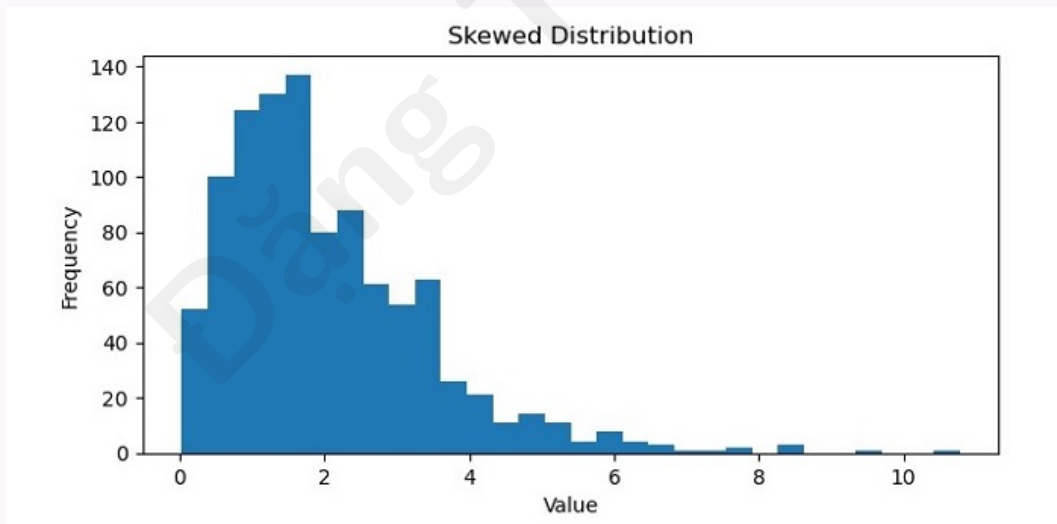
Gamma ngẫu nhiên

```
import numpy as np
import matplotlib.pyplot as plt

data = np.random.gamma(2, 1, 1000)

plt.figure(figsize=(7.5, 3.5))
plt.hist(data, bins=30)
plt.xlabel('Value')
plt.ylabel('Frequency')
plt.title('Skewed Distribution')
plt.show()
```

Output:



5.3 Phân phối đều (Uniform)

Uniform distribution

Mọi kết quả có **xác suất bằng nhau**; không có cụm quanh một giá trị. Hữu ích làm chuẩn so sánh hoặc sinh số ngẫu nhiên không thiên lệch.

PDF liên tục

$$f(x) = \begin{cases} 1 & \text{nếu } a \leq x \leq b \\ 0 & \text{ngược lại} \end{cases}$$

Mean: $\frac{a+b}{2}$; variance: $\frac{(b-a)^2}{12}$.

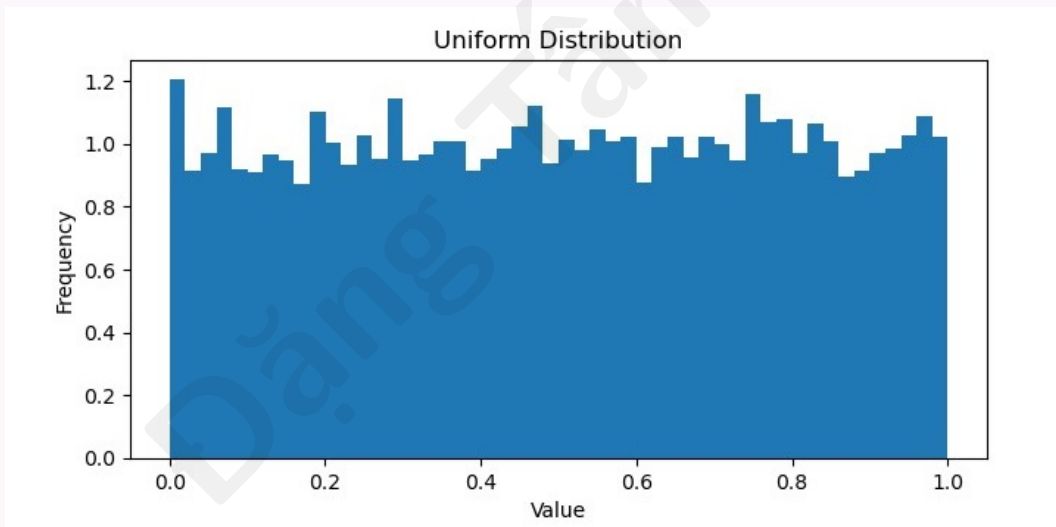
Uniform trên $[0, 1]$

```
import numpy as np
import matplotlib.pyplot as plt

uniform_data = np.random.uniform(low=0, high=1, size=10000)

plt.figure(figsize=(7.5, 3.5))
plt.hist(uniform_data, bins=50, density=True)
plt.xlabel('Value')
plt.ylabel('Frequency')
plt.title('Uniform Distribution')
plt.show()
```

Output:



5.4 Phân phối hai đỉnh (Bimodal)

Bimodal distribution

Phân phối có **hai mode** (hai đỉnh), thường tách bởi vùng thấp (thung lũng) ở giữa — có thể phản ánh hai tiểu quần thể.

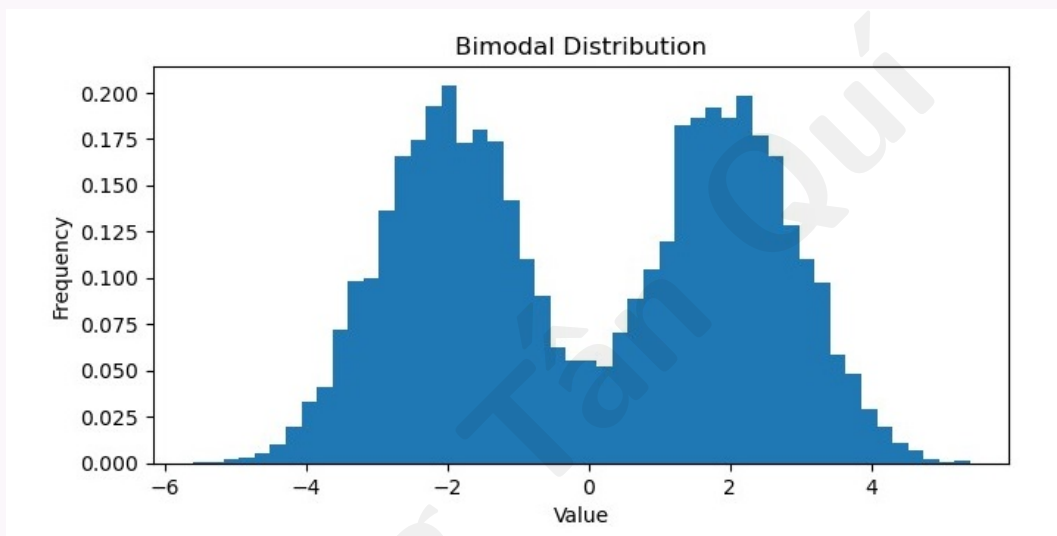
Hai cụm Gaussian

```
import numpy as np
import matplotlib.pyplot as plt
```

```
bimodal_data = np.concatenate((
    np.random.normal(loc=-2, scale=1, size=5000),
    np.random.normal(loc=2, scale=1, size=5000)
))

plt.figure(figsize=(7.5, 3.5))
plt.hist(bimodal_data, bins=50, density=True)
plt.xlabel('Value')
plt.ylabel('Frequency')
plt.title('Bimodal Distribution')
plt.show()
```

Output:



6 Skewness and Kurtosis — Độ lệch và độ nhọn

Skewness (độ lệch)

Skewness đo **độ bất đối xứng** của phân phối.

- Dương: đuôi dài bên phải.
- Âm: đuôi dài bên trái.
- Bằng 0: đối xứng (như phân phối chuẩn lý tưởng).

Kurtosis (độ nhọn)

Kurtosis đo **độ nhọn** của phân phối so với chuẩn.

- Kurtosis cao: đỉnh nhọn hơn, đuôi nặng hơn chuẩn.
- Kurtosis thấp: đỉnh dẹt hơn, đuôi nhẹ hơn.
- Bằng 0: gần chuẩn.

Ảnh hưởng tới ML

Phân phối lệch mạnh có thể cần biến đổi dữ liệu hoặc phương pháp phi tham số. Kurtosis cao có thể cần mô hình hoặc ước lượng robust hơn.

6.1 Ví dụ Python (SciPy)

skew() và kurtosis()

```
import numpy as np
from scipy.stats import skew, kurtosis

data = np.random.normal(0, 1, 1000)

skewness = skew(data)
kurt_val = kurtosis(data)

print('Skewness:', skewness)
print('Kurtosis:', kurt_val)
```

Output (ví dụ, gần 0 với mẫu chuẩn):

```
Skewness: -0.04119418903611285
Kurtosis: -0.1152250196054534
```

7 Bias and Variance — Độ lệch và phương sai mô hình

Hai nguồn sai số chính

Bias và variance mô tả nguồn lỗi trong dự đoán của mô hình ML. **Bias**: lỗi do **đơn giản hoá quá mức** quan hệ giữa feature và đầu ra. **Variance**: lỗi do **nhạy quá** với dao động nhỏ trong dữ liệu huấn luyện. Mục tiêu: giảm cả hai để dự đoán tốt trên dữ liệu chưa thấy.

Ba thành phần lỗi

Bias, variance và **irreducible error** (nhiều không giảm được). Có **đánh đổi** bias–variance: giảm bias thường tăng variance và ngược lại.

7.1 Bias là gì?

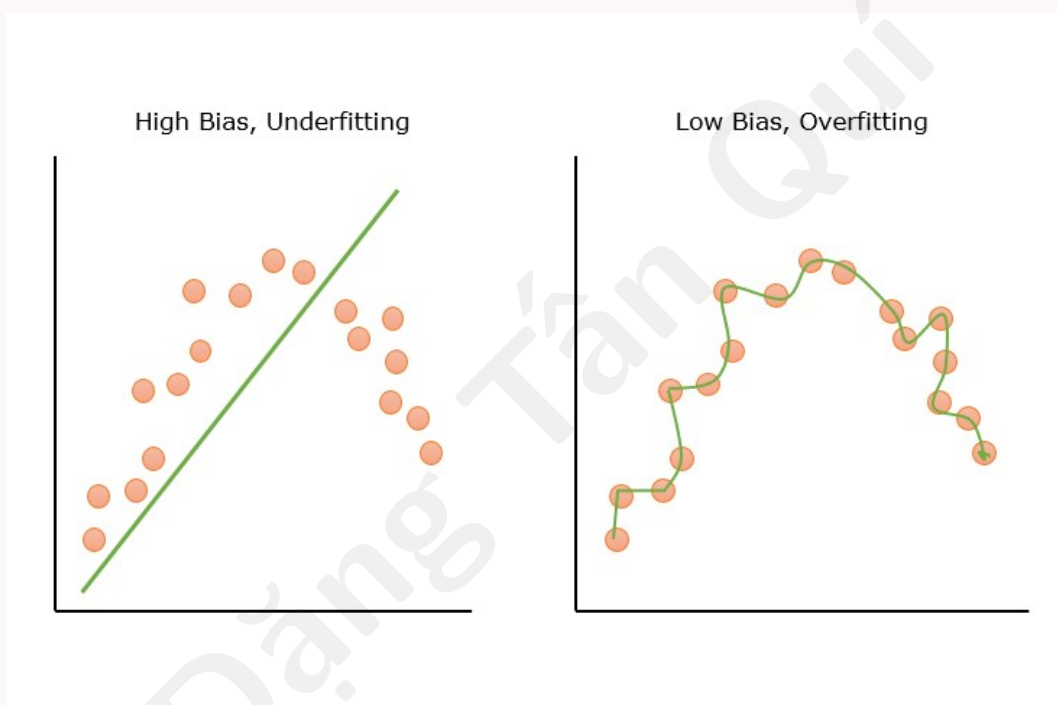
Bias

Bias được tính là chênh lệch giữa **dự đoán trung bình** và **giá trị thực**. Trong ML, bias (sai số hệ thống) xảy ra khi mô hình **giả định sai** về dữ liệu.

High bias vs low bias

- **High bias**: mô hình quá đơn giản, không khớp train lẫn test $\Rightarrow$ **underfitting**.
- **Low bias**: bắt được pattern ẩn; nếu quá phức tạp dễ dẫn tới variance cao và overfitting.

Minh họa bias cao / thấp



Ví dụ mô hình

Hồi quy tuyến tính trên dữ liệu phi tuyến $\Rightarrow$ bias cao. Linear/logistic regression thường bias cao hơn; cây quyết định, kNN, SVM thường bias thấp hơn (linh hoạt hơn).

7.2 Variance là gì?

Variance

Trong thống kê, variance đo độ trải của tập số quanh mean. Trong ML, variance là **biến thiên dự đoán** khi huấn luyện trên các tập dữ liệu hơi khác nhau.

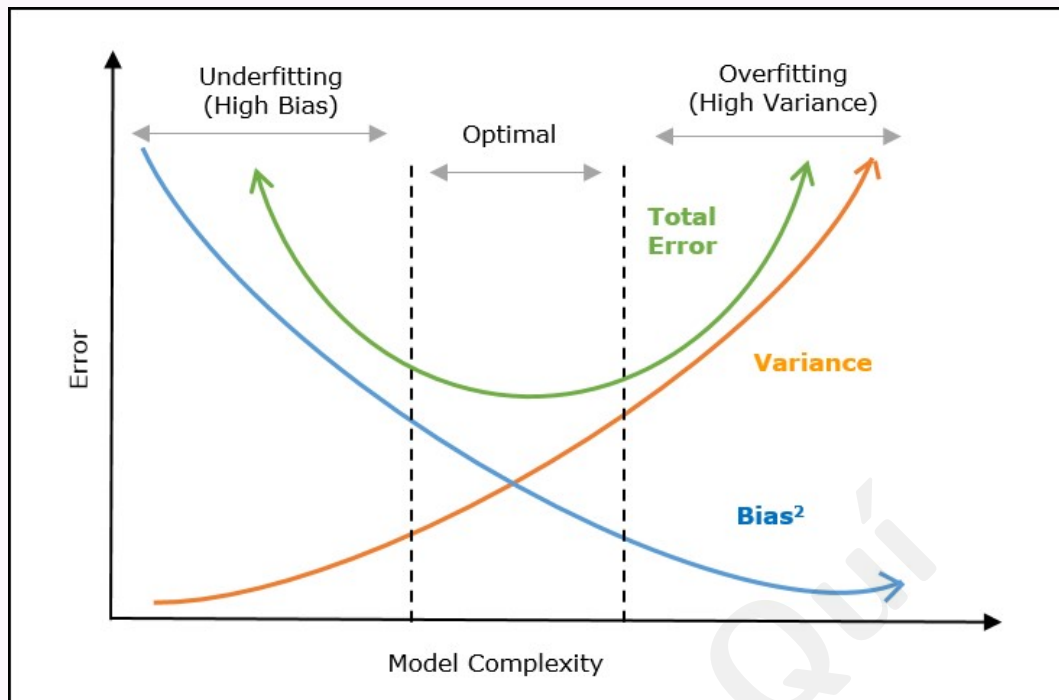
High variance vs low variance

- **High variance:** khớp cả nhiều trên train, kém trên test $\Rightarrow$ **overfitting**.
- **Low variance:** ít thay đổi giữa train/test; mô hình quá đơn giản có thể kèm bias cao.

Minh họa variance cao / thấp**7.3 Đánh đổi Bias–Variance****Cân bằng**

Tăng độ phức tạp mô hình $\Rightarrow$ bias giảm, variance tăng. Giảm độ phức tạp $\Rightarrow$ bias tăng, variance giảm. Cần điểm tối ưu sao cho **tổng lỗi dự đoán** nhỏ nhất.

Đồ thị đánh đổi Bias–Variance



Trục X: độ phức tạp mô hình; trục Y: lỗi dự đoán. Tổng lỗi = bias + variance (+ irreducible); vùng tối ưu là chỗ cân bằng hai thành phần.

Biểu thức toán học

$$\text{Error} = \text{bias}^2 + \text{variance} + \text{irreducible error}.$$

Giảm lỗi dự đoán bằng cách chọn độ phức tạp cân bằng bias và variance.

Kỹ thuật cân bằng

Giảm bias cao: mô hình phức tạp hơn; thêm feature; giảm regularization.

Giảm variance cao: regularization; đơn giản hoá mô hình; thêm dữ liệu; cross-validation.

7.4 Ví dụ Python

High bias — Linear Regression

```
import numpy as np
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

data = fetch_california_housing()
X, y = data.data, data.target
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

lr = LinearRegression()
lr.fit(X_train, y_train)

train_mse = mean_squared_error(y_train, lr.predict(X_train))
test_mse = mean_squared_error(y_test, lr.predict(X_test))
print("Training MSE:", train_mse)
print("Testing MSE:", test_mse)
```

Output (mức MSE phụ thuộc dữ liệu; train và test đều cao, gợi ý bias đáng kể so với mô hình phức tạp hơn).

Low bias, high variance — Polynomial Regression

```
from sklearn.preprocessing import PolynomialFeatures

poly = PolynomialFeatures(degree=2)
X_train_poly = poly.fit_transform(X_train)
X_test_poly = poly.transform(X_test)

pr = LinearRegression()
pr.fit(X_train_poly, y_train)

train_mse = mean_squared_error(y_train, pr.predict(X_train_poly))
test_mse = mean_squared_error(y_test, pr.predict(X_test_poly))
print("Training MSE:", train_mse)
print("Testing MSE:", test_mse)
```

Train MSE thường **thấp hơn nhiều** test MSE $\Rightarrow$ overfitting / variance cao.

Giảm variance — Ridge Regression

```
from sklearn.linear_model import Ridge

ridge = Ridge(alpha=1)
ridge.fit(X_train_poly, y_train)

train_mse = mean_squared_error(y_train, ridge.predict(X_train_poly))
test_mse = mean_squared_error(y_test, ridge.predict(X_test_poly))
print("Training MSE:", train_mse)
print("Testing MSE:", test_mse)
```

Test MSE thường cải thiện so với đa thức không regularization — variance giảm, bias tăng nhẹ.

Tối ưu α — GridSearchCV

```
from sklearn.model_selection import GridSearchCV
```

```
param_grid = {'alpha': np.logspace(-3, 3, 7)}
ridge_cv = GridSearchCV(Ridge(), param_grid, cv=5)
ridge_cv.fit(X_train_poly, y_train)

train_mse = mean_squared_error(y_train, ridge_cv.predict(X_train_poly))
test_mse = mean_squared_error(y_test, ridge_cv.predict(X_test_poly))
print("Training MSE:", train_mse)
print("Testing MSE:", test_mse)
```

Chọn α qua CV để cân bằng bias–variance tốt hơn.

8 Hypothesis — Giả thuyết trong học máy

Hypothesis (giả thuyết)

Trong ML, giả thuyết là **lời giải thích hoặc giải pháp đề xuất** cho bài toán — giả định tạm thời có thể kiểm tra bằng dữ liệu. Trong học có giám sát, giả thuyết chính là **mô hình** thuật toán học để dự đoán trên dữ liệu mới.

Ví dụ đời thường

“Giá nhà tỷ lệ thuận với diện tích sàn” — một giả thuyết có thể kiểm tra bằng dữ liệu.

8.1 Giả thuyết trong Machine Learning

Hàm giả thuyết

Giả thuyết thường là hàm ánh xạ đầu vào sang dự đoán:

$$h(x) = \hat{y}$$

với x là đầu vào, $\hat{y}$ là giá trị dự đoán. Mục tiêu ML: tìm giả thuyết **tổng quát hoá tốt** trên dữ liệu chưa thấy.

8.2 Hàm giả thuyết h

Hypothesis function

Hàm giả thuyết nhận đầu vào và trả đầu ra. Hồi quy tuyến tính đơn biến:

$$h(x) = w_0 + w_1x$$

w_0, w_1 là tham số (trọng số); x là feature.

Hồi quy tuyến tính đa biến:

$$h(x) = w_0 + w_1x_1 + \cdots + w_nx_n$$

Huấn luyện

Quá trình tìm giá trị tối ưu cho tham số sao cho **hàm mất mát** (cost) nhỏ nhất.

8.3 Không gian giả thuyết $\mathcal{H}$ **Hypothesis space**

Tập mọi giả thuyết có thể có gọi là **không gian giả thuyết**. Với hồi quy tuyến tính, không gian gồm mọi hàm tuyến tính có thể. Thuật toán tìm giả thuyết “khớp” nhất trong không gian này — gọi là **huấn luyện / học**.

8.4 Hai loại giả thuyết (kiểm định thống kê)**Null hypothesis (H_0)**

Giả thuyết **mặc định**: không có quan hệ giữa feature và biến mục tiêu. Trong ML ta thường cố **bác bỏ** H_0 nếu $p\text{-value} < \alpha$ (mức ý nghĩa).

Alternative hypothesis (H_1)

Giả thuyết **đối lập** H_0 : có quan hệ có ý nghĩa giữa đầu vào và đầu ra. Bác bỏ $H_0 \Rightarrow$ chấp nhận H_1 .

8.5 Kiểm định giả thuyết trong ML**Các bước**

1. Phát biểu H_0 và H_1 .
2. Chọn mức ý nghĩa α (thường 0,05 hoặc 0,01).
3. Tính thống kê kiểm định ($t, z, \dots$).
4. Tính $p\text{-value}$; nếu $p < \alpha$ thì bác bỏ H_0 .
5. Kết luận: quan hệ có ý nghĩa hay không.

8.6 Tìm giả thuyết tốt nhất**Tối ưu hoá**

Huấn luyện = điều chỉnh tham số để tối thiểu hóa loss (sai khác giữa $\hat{y}$ và y). Kỹ thuật như **gradient descent** tìm w tối ưu.

Ví dụ hồi quy tuyến tính, cost (MSE):

$$J(w) = \frac{1}{2n} \sum_{i=1}^n (h(x_i) - y_i)^2$$

Mục tiêu: w làm $J(w)$ nhỏ nhất $\Rightarrow$ giả thuyết tốt nhất trong lớp hàm đã chọn.

8.7 Tính chất giả thuyết tốt

Một giả thuyết / mô hình tốt nên

- **Generalization**: dự đoán tốt trên dữ liệu mới.
- **Simplicity**: đơn giản, dễ diễn giải.
- **Robustness**: chịu được nhiễu và outlier.
- **Scalability**: xử lý được dữ liệu lớn.

Các học thuật toán sinh giả thuyết

Linear/logistic regression, cây quyết định, SVM, mạng nơ-ron, ...

Sau huấn luyện cần **đánh giá** trên tập validation hoặc cross-validation trước khi triển khai thực tế.